

Lab 0 Exam

准备工作：创建并切换到 lab0-exam 分支

请在**自动初始化分支后**，在开发机依次执行以下命令：

```
$ cd ~/学号
$ git fetch
$ git checkout lab0-exam
```

初始化的 lab0-exam 分支在 msrc 目录下放置了完成本题所需的代码。

题目背景

在管理一个大规模程序项目的代码时，我们常会遇到这样的情况：代码存在多种构建规则，各规则之间存在共同依赖，而共同部分以外的依赖各自不同。在每次构建代码时，我们希望尽可能复用已有的编译结果，即仅对依赖项发生改变的部分重新进行编译，而不重新编译依赖项未发生改变的部分，以减少不必要的编译开销。

现在给定一些已经编写好的代码，需要你为其编写 Makefile，进行如上所述情况的代码构建管理。

此外，编写的 Makefile 还需实现常用规则 clean 与 run，分别用于：

- 清理生成的可执行程序、临时文件；
- 按照要求，依次运行编译好的程序。

文件说明

自动初始化完成后，题目有关代码将被放置在 msrc 目录下。msrc 的目录结构如下所示：

```
*
|- calc1.c
|- calc2.c
|- casegen.c
|- common.h
|- main.c
|- Makefile
|- makefile.inc
└- post_calc.c
```

- 我们需要使用 calc1.c 与 calc2.c 对同一组函数的不同定义，利用共同的依赖文件 post_calc.c, common.h 和主函数定义 main.c 编译出两个版本的程序 ver1 与 ver2。
- casegen.c 是输入生成程序的源码，由其编译得到输入生成程序 casegen。
- makefile.inc 包含了两个变量定义 INPUT_FILE 与 CASE_NUM，分别用于指定生成的输入文件名称，以及输入文件中包含的输入数据数。
- Makefile 需要你根据题目要求进行编写。

题目要求

为了保证评测的正常进行，请不要在 msrc 目录下添加其他文件。

对于涉及编译程序的构建规则，请保证**在其执行后，在文件目录下新增的文件，仅有可执行的程序。**假如你的编译过程中有中间产物（如 `.o` 文件），你需要保证这些临时文件在可执行程序生成后被移除，否则会影响评测的正常进行。

要求你补全题目提供的 Makefile，使得其具备的构建规则满足以下要求：

- `casegen`：编译生成输入生成程序 `casegen`；
- `ver1`：使用共同依赖，以及依赖 `calc1.c`，编译生成程序 `ver1`；
- `ver2`：使用共同依赖，以及依赖 `calc2.c`，编译生成程序 `ver2`；
- `all`：编译生成上述所有程序 `casegen`, `ver1`, `ver2`；
- `run`：先执行 `casegen` 获取输入，再分别执行 `ver1`, `ver2` 获取输出；
- `clean`：清理编译好的可执行文件 `casegen`, `ver1`, `ver2`，以及运行过程中产生的所有后缀为 `.txt` 的临时文件。

此外，你需要保证 `run` 为 Makefile 的默认规则，即执行命令 `make` 与执行 `make run` 效果等价。

需要注意的是，`run` 和 `clean` 规则为伪规则，即不对应一个要实际生成的文件。在编写 Makefile 时，你需要确保 Makefile 对这两个规则的实现符合伪规则的预期行为。举例而言，当前目录下如果创建了一个名为 `clean` 的文件，则其不应影响 `make clean` 的正常执行。

作为额外要求，我们希望 Makefile 中规则的依赖项被规范填写，即对每条规则定义 `<rule>:` `<dependency>`，其中的 `<dependency>` 部分均包含当前规则的所有依赖项。

程序 `casegen`, `ver1`, `ver2` 的使用方法如下。有关输入参数可通过 `makefile.inc` 中的定义得到，并在 Makefile 中引用：

```
casegen <case_num> <input_file>
ver1 <input_file>
ver2 <input_file>
```

样例输出

此处仅介绍各程序的运行行为。各 Makefile 规则的预期行为详见题目要求。

- 程序 `casegen` 将根据输入参数 `<case_num>`，在名为 `<input_file>` 的输入文件中，生成 `<case_num>` 组输入数据。
- 程序 `ver1` 将从 `<input_file>` 中读入输入数据，在文件 `ver1_temp{i}.txt` 中写入第 i 组输入的输出数据，并在标准输出中显示一段文本。
- `ver2` 的行为与 `ver1` 基本相同，唯输出文件的名称变为 `ver2_temp{i}.txt`。

提交评测 & 评测标准

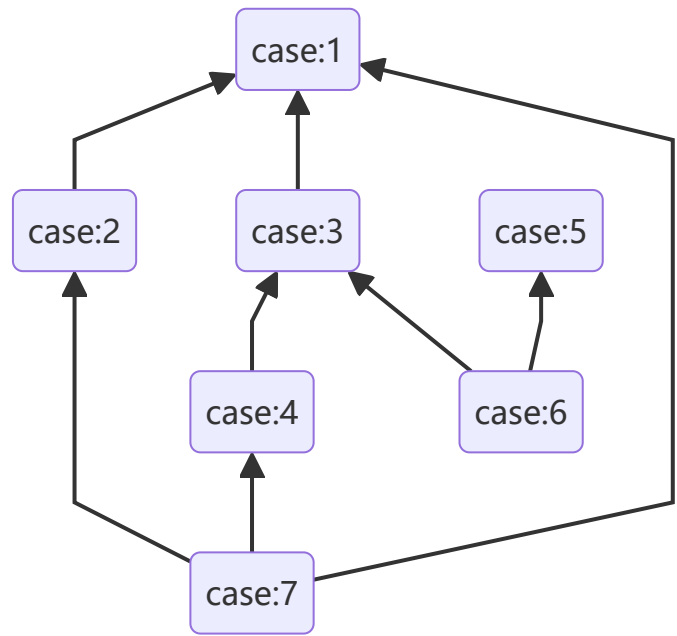
请在开发机中执行下列命令后，**在课程网站上提交评测。**

```
$ cd ~/学号/
$ git add -A
$ git commit -m "message" # 请将 message 改为有意义的信息
$ git push
```

具体要求和分数分布如下：

测试点序号	评测内容	分值
1	测试 <code>make ver1</code> , <code>make ver2</code> 及编译所得程序的正确性	20 分
2	测试 <code>make all</code>	20 分
3	测试 <code>make run</code>	20 分
4	测试只执行 <code>make</code> 的行为	10 分
5	测试 <code>make clean</code>	10 分
6	测试 <code>clean</code> , <code>run</code> 是否被正确设为伪规则	10 分
7	测试 Makefile 的依赖项是否被规范填写	10 分

测试点依赖关系如下（箭头方向表示依赖方向）：



测试样例补充说明：

- 对于测试点7，若你的程序未能正确构建 Makefile 中的依赖关系，则报错信息会显示“运行时遇到错误 exit-code=1”，正确则会生成正常的评测信息：“ACCEPTED: Dependency is set correctly!!”。
- 我们通过将你编写的 Makefile 放入测试样例提供的 `msrc` 目录下进行测试。因此，你对 `msrc` 目录下其他文件的更改均不会生效。
- `makefile.inc` 中变量的定义，可能会因测试样例不同而产生变化。
- 保证 `makefile.inc` 中定义的样例数大于1个，小于100个。
- 保证提供的测试样例中，所有输入、输出文件均以 `.txt` 作为文件后缀。
- 测试样例保证提供的程序可以正常编译，并表现出题目预期的行为。

提示：

- Makefile 中的伪规则，通过建立 `.PHONY` 规则，并在其依赖列表中，加入需要设为伪规则的规则来实现。
- Makefile 中定义的规则，可以作为另一规则的依赖项。

- 如果你对 Makefile 的编写仍有疑惑，欢迎随时参阅指导书中 Makefile 有关的内容。
- `rm <file_name>` 指令可以用于移除文件 `<file_name>`。 `<file_name>` 支持使用通配符，例如 `rm *.out` 代表删除当前目录下所有后缀为 `.out` 的文件。
- `rm` 指令在删除不存在文件时会产生报错，这一报错可以通过加入 `-f` 参数抑制，例如 `rm -f *.out`。